DOCUMENTAÇÃO TÉCNICA • VERSÃO 1.0

SentinelaWeb

Plataforma de Monitoramento e Controle de Endemias

Data: Março 2026

Stack: React + TypeScript + Supabase + PostgreSQL

Ambiente: PWA — Web + Mobile (field agents)

Status: Produção

Índice

1. **Visão Geral do Projeto**
   - 1.1 Objetivo
   - 1.2 Público-alvo
   - 1.3 Stack tecnológica
2. **Arquitetura**
   - 2.1 Visão geral de camadas
   - 2.2 Fluxo por drone
   - 2.3 Fluxo manual / vistoria
   - 2.4 Multitenancy
3. **Banco de Dados**
   - 3.1 Tabelas principais
   - 3.2 Tabelas de SLA
   - 3.3 Tabelas de Drone e Risco
   - 3.4 Módulo Vistoria de Campo (e-VISITA PNCD)
   - 3.5 Centro de Notificações de Casos
   - 3.6 Tabelas auxiliares
   - 3.7 Views e RPCs
   - 3.8 Triggers automáticos
   - 3.9 RLS — Resumo de segurança
4. **API (src/services/api.ts)**
   - 4.1 Namespaces e métodos
5. **Frontend**
   - 5.1 Hooks React Query
   - 5.2 Páginas
   - 5.3 Rotas
   - 5.4 Componentes de domínio
6. **Módulos Funcionais**
   - 6.1 SLA Operacional
   - 6.2 Risco Pluviométrico
   - 6.3 Vistoria de Campo
   - 6.4 Centro de Notificações
   - 6.5 Análise IA pós-voo
   - 6.6 Canal Cidadão
   - 6.7 PWA e Suporte Offline

7. **Segurança**
   7.1 Row Level Security (RLS)
   7.2 RBAC — Papéis
   7.3 LGPD
8. **Histórico de Migrations**

1. Visão Geral do Projeto

1.1 Objetivo

O **SentinelaWeb** é uma plataforma SaaS destinada a prefeituras para monitoramento, inspeção e controle de focos de dengue, chikungunya e zika. O sistema combina:

- **Voos com drone** — captura de imagens aéreas, processadas por pipeline Python + YOLO para detecção automática de focos
- **Vistoria terrestre (e-VISITA PNCD)** — agentes visitam imóveis imóvel a imóvel seguindo o padrão do Programa Nacional de Controle da Dengue
- **Centro de Notificações** — unidades de saúde registram casos confirmados ou suspeitos, cruzados automaticamente com focos identificados em campo
- **Canal Cidadão** — denúncias públicas via QR Code sem necessidade de login
- **SLA e auditoria** — controle de prazos de atendimento por prioridade, região e período

1.2 Público-alvo

PERFIL	FUNÇÃO
Admin	Gestores de TI e plataforma. Acesso total ao sistema
Supervisor	Gestores municipais. Visão de equipe, planejamentos, relatórios
Operador	Agentes de campo. Vistoria de imóveis, inspeção de levantamentos
Notificador	Funcionários de UBS/UPA/hospitais. Registro de casos
Cidadão	Acesso público para denúncias via QR Code

1.3 Stack Tecnológica

CAMADA	TECNOLOGIA
Frontend	React 18 + TypeScript + Vite
UI	shadcn/ui + Tailwind CSS
Estado servidor	@tanstack/react-query v5
Roteamento	React Router v6 com lazy loading
Mapas	Leaflet + react-leaflet + leaflet-heat
Banco de dados	Supabase (PostgreSQL 15 + PostGIS)
Auth	Supabase Auth (JWT)
Storage	Cloudinary (evidências fotográficas)
PDF	jspdf + jspdf-autotable
E-mail	Resend (relatórios automáticos)
Push	Web Push API + Supabase Edge Functions
Offline	IndexedDB + Workbox PWA
Pipeline drone	Python + ExifTool + YOLO + Supabase

2. Arquitetura

2.1 Visão Geral de Camadas

```
┌──────────────────────────────────────────────────────────────┐
│                          FRONTEND                            │
│  React + TypeScript + Vite                                   │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐ │
│  │   Pages      │  │ Components │  │  Hooks (React Query)  │ │
```

```
│  │ (39 pages) │  │ (132+ comp)│  │  (31 hooks)          │ │
│  └──────┬──────┘  └──────┬──────┘  └──────────┬──────────┘ │
│         └────────────────┴────────────────────┘            │
│                          │                                 │
│              src/services/api.ts (único ponto de acesso)   │
└─────────────────────────────┬──────────────────────────────┘
                              │ supabase-js
┌─────────────────────────────▼──────────────────────────────┐
│                        SUPABASE                            │
│  PostgreSQL 15 + PostGIS + Auth + Storage                  │
│                                                            │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│  │ Tables   │  │  Views   │  │  RPCs    │  │ Triggers │  │
│  │ (37 tab.)│  │ (5 views)│  │(~15 RPC) │  │ (8 trig.)│  │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
│  RLS habilitado em todas as tabelas                        │
└────────────────────────────────────────────────────────────┘
```

2.2 Fluxo por Drone

```
Planejamento (drone) → Levantamento → Voo (Python)
    → ExifTool (EXIF) → YOLO (detecção)
    → levantamento_itens (focos identificados)
    → Cloudinary (imagens evidenciadas)
    → Operador valida em campo → Plano de ação → SLA
```

2.3 Fluxo Vistoria Terrestre (e-VISITA PNCD)

```
Agente → OperadorInicioTurno → seleciona atividade
    → OperadorListaImoveis → escolhe imóvel
    → OperadorFormularioVistoria (5 etapas):
        Etapa 1: moradores + GPS checkin
        Etapa 2: sintomas → trigger cria caso_notificado se qtd > 0
        Etapa 3: depósitos A1-E (inspecionados / com focos)
        Etapa 4: tratamento (eliminação + larvicida)
        Etapa 5: riscos social/sanitário/vetorial → FINALIZAR
    → vistoria salva no banco (status = visitado)
```

2.4 Multitenancy

Todo acesso ao banco é filtrado por `cliente_id` (cada prefeitura é um cliente independente).

```
// Padrão obrigatório em TODOS os hooks
const { clienteId } = useClienteAtivo();

// Padrão obrigatório em TODOS os métodos de api.ts
.eq('cliente_id', clienteId)   // ou via !inner join
```

O RLS no PostgreSQL aplica uma segunda camada de isolamento, garantindo que mesmo queries sem filtro `cliente_id` no código retornem apenas dados do cliente autenticado.

3. Banco de Dados

3.1 Tabelas Principais

`clientes`

Representa uma prefeitura (tenant raiz).

COLUNA	TIPO	DESCRIÇÃO
id	uuid PK	Identificador
nome	text	Nome da prefeitura
slug	text	Identificador URL
cnpj	text	CNPJ
latitude_centro	float8	Centro geográfico do município
longitude_centro	float8	Centro geográfico do município

bounds	jsonb	Bounds do mapa
ativo	boolean	Status

`usuarios`

Usuários autenticados vinculados a um cliente.

COLUNA	TIPO	DESCRIÇÃO
id	uuid PK	
auth_id	uuid	Referência ao Supabase Auth
nome	text	Nome do usuário
email	text	E-mail
cliente_id	uuid FK	Prefeitura do usuário

`papeis_usuarios`

RBAC — papéis por usuário.

PAPEL	PERMISSÕES
`admin`	Acesso total ao sistema
`supervisor`	Gestão de equipe e planejamentos
`usuario`	Operador padrão
`operador`	Agente de campo (vistoria)
`notificador`	Registro de casos (UBS/hospital)

`regioes`

Divisões geográficas do município.

COLUNA	TIPO
id	uuid PK
cliente_id	uuid FK
nome	text

geometria	jsonb

`planejamento`

Organização do trabalho de campo.

COLUNA	TIPO	DESCRIÇÃO
id	uuid PK	
cliente_id	uuid FK	
regiao_id	uuid FK	
titulo	text	
tipo	text	`drone` | `manual`
tipo_levantamento	text	
status	text	`rascunho` | `ativo` | `concluido`

`levantamentos`

Execução associada a um planejamento.

COLUNA	TIPO	DESCRIÇÃO
id	uuid PK	
cliente_id	uuid FK	
planejamento_id	uuid FK	
titulo	text	
data_voo	date	
tipo_entrada	text	`DRONE` | `MANUAL`
config_fonte	text	`supabase` | `local_json` (auditoria de fallback)

`levantamento_itens`

Cada foco ou problema identificado.

COLUNA	TIPO	DESCRIÇÃO

id	uuid PK	
levantamento_id	uuid FK	
tipo	text	Classificação YOLO ou manual
descricao	text	
latitude / longitude	float8	Coordenadas GPS
score_final	float8	Confiança YOLO (0–1 normalizado)
prioridade	text	`Crítico` | `Urgente` | `Alta` | `Média` | `Baixa`
status_atendimento	text	`pendente` | `em_atendimento` | `resolvido`
acao_aplicada	text	
observacao_atendimento	text	
checkin_em	timestamptz	GPS check-in do operador
checkin_latitude/longitude	float8	
payload	jsonb	Dados extras (caso_notificado_proximidade, etc.)

> ***Score YOLO**: pode ser gravado como* `0–1` *ou* `0–100` *pelo pipeline Python. Sempre normalizar:* `raw > 1 ? raw / 100 : raw` *.*

`levantamento_item_evidencias`

Fotos tiradas pelo operador ao verificar o foco.

COLUNA	TIPO
levantamento_item_id	uuid FK
url	text (Cloudinary)
tipo	text

`levantamento_item_status_historico`

Auditoria de mudanças de status de cada item.

`plano_acao_catalogo`

Catálogo de ações corretivas por tipo de item, configurável por cliente.

`levantamento_item_recorrencia`

Agrupa itens recorrentes no mesmo endereço — identificação de focos persistentes.

3.2 Tabelas de SLA

`sla_operacional`

Prazos de atendimento associados a levantamento_itens.

COLUNA	TIPO	DESCRIÇÃO
id	uuid PK	
cliente_id	uuid FK	
levantamento_item_id	uuid FK	
prioridade	text	
prazo_horas	int	Calculado por `sla_calcular_prazo_final()`
prazo_final	timestamptz	Considerando feriados e horário comercial
status	text	`em_prazo` | `vencido` | `resolvido` | `escalado`
escalonado_em	timestamptz	

`sla_config`

Configuração de SLA por cliente (base para cálculo).

`sla_config_regiao`

Override de SLA por região geográfica.

`sla_feriados`

Feriados cadastrados por cliente (nacionais + municipais) para cálculo de prazo em horário comercial.

`sla_config_audit`

Auditoria de todas as alterações de configuração de SLA.

Regras de SLA:

PRIORIDADE	SLA BASE	REDUÇÕES POSSÍVEIS
Crítico / Urgente	4h	−30% se risco "Muito Alto"
Alta	12h	−20% se persistência > 3 dias
Moderada / Média	24h	−10% se temperatura > 30°C
Baixa / Monitoramento	72h	—
Mínimo absoluto	**2h**	Sempre

3.3 Tabelas de Drone e Risco

`voos`

Voos de drone realizados, vinculados a planejamentos.

COLUNA	TIPO
id	uuid PK
planejamento_id	uuid FK
piloto_id	uuid FK usuarios
data_voo	date
status	text

`sentinela_drone_risk_config`

Configuração da política de risco do drone por cliente (scoring YOLO, thresholds, SLA por prioridade).

`sentinela_yolo_class_config`

Mapeamento das classes YOLO para itens do sistema (item_key → risco, peso, ação).

`sentinela_yolo_synonym`

Sinônimos para classes YOLO — permite flexibilidade no pipeline Python.

`sentinela_risk_policy` e subentidades

Política de risco **pluviométrico** (bins de chuva, fatores de temperatura/vento, persistência).

> ⚠️ ***Distinção importante:*** `sentinela_risk_policy` *= risco pluviométrico.* `sentinela_drone_risk_config` *= risco de drone/YOLO. São sistemas completamente separados.*

`pluvio_risco`

Score de risco pluviométrico calculado diariamente por região.

`pluvio_operacional_run` / `pluvio_operacional_item`

Resultado de rodadas de análise operacional de risco de chuva.

3.4 Módulo Vistoria de Campo (e-VISITA PNCD)

Inspirado no e-VISITA Endemias da SES/MS. Permite ao agente registrar vistorias domiciliar por domiciliar.

`imoveis`

Cadastro de imóveis visitáveis.

COLUNA	TIPO	DESCRIÇÃO
id	uuid PK	

cliente_id	uuid FK	
regiao_id	uuid FK	Opcional
tipo_imovel	text	`residencial` | `comercial` | `terreno` | `ponto_estrategico`
logradouro	text	
numero	text	
bairro / quarteirao	text	
latitude / longitude	float8	
ativo	boolean	

`vistorias`

Cada visita de um agente a um imóvel em um ciclo.

COLUNA	TIPO	DESCRIÇÃO
id	uuid PK	
imovel_id / agente_id	uuid FK	
ciclo	int	1–6 (ciclos bimestrais)
tipo_atividade	text	`tratamento` | `pesquisa` | `liraa` | `ponto_estrategico`
status	text	`pendente` | `visitado` | `fechado` | `revisita`
moradores_qtd	int	
gravidas / idosos / criancas_7anos	boolean	Grupos vulneráveis
lat_chegada / lng_chegada	float8	GPS no checkin
checkin_em	timestamptz	

`vistoria_depositos`

Inspeção por tipo de depósito — padrão PNCD (um registro por tipo por vistoria).

TIPO	DESCRIÇÃO
A1	Caixa d'água elevada
A2	Outro armazenamento de água

B	Pequenos depósitos móveis
C	Depósitos fixos
D1	Pneus e materiais rodantes
D2	Lixo (rec. plást., latas, suc.)
E	Depósitos naturais

Campos: `qtd_inspecionados`, `qtd_com_focos`, `qtd_eliminados`, `usou_larvicida`, `qtd_larvicida_g`.

`vistoria_sintomas`

Sintomas registrados durante a visita. **Sem dados pessoais (LGPD)** — apenas contagem.

COLUNA	TIPO
febre / manchas_vermelhas / dor_articulacoes / dor_cabeca	boolean
moradores_sintomas_qtd	int
gerou_caso_notificado_id	uuid FK (auto pelo trigger)

`vistoria_riscos`

Fatores de risco identificados pelo agente.

CATEGORIA	CAMPOS
Social	menor_incapaz, idoso_incapaz, dep_quimico, risco_alimentar, risco_moradia
Sanitário	criadouro_animais, lixo, residuos_organicos, residuos_quimicos, residuos_medicos
Vetorial	acumulo_material_organico, animais_sinais_lv, caixa_destampada, outro_risco_vetorial

---

3.5 Centro de Notificações de Casos

Integração entre unidades de saúde e focos identificados em campo.

`unidades_saude`

Postos de saúde, UPAs e hospitais vinculados ao cliente.

COLUNA	TIPO
tipo	text
latitude / longitude	float8

`casos_notificados`

Casos de dengue/chikungunya/zika registrados. **Sem dados pessoais (LGPD).**

COLUNA	TIPO	DESCRIÇÃO
doenca	text	`dengue` | `chikungunya` | `zika` | `suspeito`
status	text	`suspeito` | `confirmado` | `descartado`
endereco_paciente	text	Endereço de residência (sem nome/CPF)
latitude / longitude	float8	Geocodificado do endereço
data_inicio_sintomas	date	

`caso_foco_cruzamento`

Vínculo automático criado por trigger entre casos e focos próximos.

COLUNA	TIPO
caso_id	uuid FK
levantamento_item_id	uuid FK
distancia_metros	float8

3.6 Tabelas Auxiliares

TABELA	PROPÓSITO
`operacoes`	Operações de atendimento com ciclo de vida em_andamento/concluido
`operacao_evidencias`	Evidências fotográficas de operações

`cliente_quotas`	Limites de uso (voos/mês, itens/mês, storage)
`push_subscriptions`	Subscrições Web Push por usuário
`yolo_feedback`	Feedback do operador sobre detecções YOLO (falso positivo)
`levantamento_analise_ia`	Resultados da triagem IA pós-voo (cluster + sumário Claude)
`tags`	Tags globais aplicáveis a itens

3.7 Views e RPCs

Views Materializadas/Regulares

VIEW	PROPÓSITO
`v_recorrencias_ativas`	Itens recorrentes no mesmo endereço (>1 ocorrência/30d)
`v_slas_iminentes`	SLAs com menos de 1h restante
`v_historico_atendimento_local`	Histórico de atendimentos por endereço
`v_cliente_uso_mensal`	Uso atual vs quota do cliente
`view_rls_rules`	Auditoria de políticas RLS (via pg_policies)

RPCs Principais

FUNÇÃO	PARÂMETROS	RETORNO
`get_meu_papel(p_auth_id)`	auth_id	papel (text)
`criar_levantamento_item_manual(...)`	dados do item	levantamento_item
`sla_calcular_prazo_final(...)`	prioridade, config, feriados	timestamptz
`sla_resolve_config(...)`	cliente_id, regiao_id, prioridade	sla_config
`marcar_slas_vencidos(p_cliente_id)`	—	void
`escalar_sla_operacional(p_id)`	—	void
`escalar_slas_iminentes(p_cliente_id)`	—	void

`avaliar_condicoes_voo(p_cliente_id, p_data)`	—	CondicaoVoo JSON
`cliente_verificar_quota(...)`	cliente_id, tipo_quota	QuotaVerificacao
`resumo_agente_ciclo(p_cliente_id, p_agente_id, p_ciclo)`	—	JSON stats
`listar_casos_no_raio(p_lat, p_lng, p_raio, p_cliente)`	—	casos_notificados[]
`denuncia_cidadao(...)`	dados da denúncia	void (SECURITY DEFINER)

3.8 Triggers Automáticos

TRIGGER	TABELA	AÇÃO
`trg_sintomas_para_caso`	`vistoria_sintomas` INSERT	Cria `casos_notificados` se `moradores_sintomas_qtd > 0`
`trg_cruzar_caso_focos`	`casos_notificados` INSERT	Cruza com `levantamento_itens` em raio 300m (PostGIS), eleva prioridade para Crítico
`trg_auto_sla_levantamento_item`	`levantamento_itens` INSERT	Cria registro em `sla_operacional` automaticamente
`trg_auto_sla_pluvio`	`pluvio_operacional_item` INSERT	Cria SLA para itens de risco pluvial
`trg_operacao_concluido`	`operacoes` UPDATE	Grava `concluido_em` ao mudar status
`trg_seed_drone_risk_config`	`clientes` INSERT	Popula configuração YOLO/drone para novo cliente
`trg_sla_config_audit`	`sla_config` UPDATE	Registra todas as alterações de configuração
`trg_status_historico_item`	`levantamento_itens` UPDATE	Registra histórico de mudanças de status

3.9 RLS — Resumo de Segurança

Todas as tabelas têm Row Level Security habilitado. O padrão de isolamento é:

```
CREATE POLICY "isolamento_por_cliente" ON nome_tabela
  USING (cliente_id IN (
    SELECT cliente_id FROM usuarios WHERE auth_id = auth.uid()
  ));
```

TABELA	ESTRATÉGIA RLS	OBS
clientes, usuarios, regioes, planejamento	`cliente_id` direto	Padrão
levantamento_itens, evidencias	`join levantamentos`	Via levantamento
sla_operacional, sla_config	`cliente_id` direto	
voos	`join planejamento`	Via planejamento
imoveis, vistorias, vistoria_sintomas	`cliente_id` direto	
vistoria_depositos, vistoria_riscos	`join vistorias`	Via vistoria
casos_notificados, unidades_saude	`cliente_id` direto	
caso_foco_cruzamento	`join casos_notificados`	Via caso
drones	Global (sem cliente_id)	Frota compartilhada
tags	Authenticated users	Dados globais

4. API — `src/services/api.ts`

Único arquivo de acesso ao banco. **Proibido** usar `supabase` diretamente em componentes ou páginas.

4.1 Namespaces e Métodos

`api.levantamentos`

MÉTODO	DESCRIÇÃO
`list(clienteId)`	Lista levantamentos do cliente
`updatePlanejamento(id, planejamentoId)`	Vincula levantamento a planejamento

`api.itens`

MÉTODO	DESCRIÇÃO
`listByLevantamento(levId)`	Itens de um levantamento
`listByCliente(clienteId)`	Todos os itens do cliente
`listByOperador(clienteId, usuarioId)`	Itens atribuídos ao operador
`listMapByCliente(clienteId)`	Versão leve para mapa
`criarManual(...)`	Cria item manualmente (RPC)
`updateAtendimento(id, payload)`	Atualiza status + ação aplicada
`registrarCheckin(id, coords)`	GPS check-in do operador
`listStatusHistorico(itemId)`	Histórico de status
`listByClienteAndPeriod(clienteId, from, to)`	Filtro por período

`api.imoveis`

MÉTODO	DESCRIÇÃO
`list(clienteId, regiaoId?)`	Lista imóveis (filtrável por região)
`create(payload)`	Cadastra novo imóvel
`update(id, payload)`	Atualiza imóvel

`api.vistorias`

MÉTODO	DESCRIÇÃO
`listByAgente(clienteId, agenteId, ciclo?)`	Vistorias do agente no ciclo

`listByImovel(imovelId)`	Histórico de vistorias do imóvel
`create(payload)`	Cria vistoria principal
`updateStatus(id, status)`	Atualiza status
`addDeposito(vistoriaId, deposito)`	Upsert de depósito (por tipo)
`addSintomas(sintomas)`	Registra sintomas
`addRiscos(riscos)`	Registra riscos identificados
`getResumoAgente(clienteId, agenteId, ciclo)`	Estatísticas do agente no ciclo (RPC)

`api.casosNotificados`

MÉTODO	DESCRIÇÃO
`list(clienteId)`	Lista casos do cliente
`create(payload)`	Registra novo caso
`updateStatus(id, status)`	Confirma ou descarta caso
`countProximoAoItem(itemId)`	Contagem de casos em 300m do foco
`listProximosAoPonto(lat, lng, clienteId, raioMetros?)`	Casos próximos a ponto
`cruzamentosDoItem(itemId)`	Cruzamentos do item com casos

`api.unidadesSaude`

MÉTODO	DESCRIÇÃO
`list(clienteId)`	Lista unidades do cliente
`create(payload)`	Cadastra unidade
`update(id, payload)`	Atualiza unidade

`api.sla`

MÉTODO	DESCRIÇÃO
`listByCliente(clienteId)`	SLAs do cliente
`listForPanel(clienteId, operadorId?)`	SLAs para painel operador

`updateStatus(id, status)`	Atualiza status do SLA
`reabrir(id)`	Reabre SLA resolvido
`verificarVencidos(clienteId)`	Marca SLAs vencidos (Edge Function)
`escalar(id)`	Escalona SLA
`pendingCount(clienteId)`	Contagem pendentes (badge)

Outros namespaces

`api.planejamentos`, `api.regioes`, `api.clientes`, `api.operacoes`, `api.map`, `api.pluvio`, `api.planoAcaoCatalogo`, `api.recorrencias`, `api.droneRiskConfig`, `api.yoloClassConfig`, `api.slaFeriados`, `api.slaIminentes`, `api.slaConfigRegiao`, `api.quotas`, `api.pushSubscriptions`, `api.condicoesVoo`, `api.yoloFeedback`, `api.analiseIa`, `api.cloudinary`, `api.admin`, `api.tags`, `api.evidenciasItem`.

---

5. Frontend

5.1 Hooks React Query

Todos os hooks ficam em `src/hooks/queries/`. Seguem o padrão:

```
export function useNomeHook(clienteId: string | null | undefined) {
  return useQuery({
    queryKey: ['chave', clienteId],
    queryFn: () => api.namespace.metodo(clienteId!),
    enabled: !!clienteId,
    staleTime: STALE.SHORT, // de src/lib/queryConfig.ts
  });
}
```

HOOK	QUERY KEY	STALETIME
`useImoveis`	`['imoveis', clienteId, regiaoId]`	LONG (10min)
`useVistorias`	`['vistorias', clienteId, agenteId, ciclo]`	SHORT (1min)
`useVistoriaResumo`	`['vistoria_resumo', clienteId, agenteId, ciclo]`	SHORT
`useCasosNotificados`	`['casos_notificados', clienteId]`	2min
`useUnidadesSaude`	`['unidades_saude', clienteId]`	10min
`useSla`	`['sla', clienteId]`	SHORT
`usePluvio`	`['pluvio_risco', clienteId]`	MEDIUM (3min)
`useMapData`	`['map_full_data', clienteId]`	MAP (10min)
`useItensOperador`	`['itens_operador', clienteId, usuarioId]`	gcTime 30min
`useCondicoesVoo`	`['condicoes_voo', clienteId]`	15min
`useClienteQuotas`	`['cliente_quotas', clienteId]`	5min

Constantes de staleTime (`src/lib/queryConfig.ts`):

CONSTANTE	VALOR
`STALE.LIVE`	0
`STALE.SHORT`	1 min
`STALE.MEDIUM`	3 min
`STALE.LONG` / `STALE.MAP`	10 min
`STALE.STATIC`	30 min
`STALE.SESSION`	Infinity

5.2 Páginas

Admin (18 páginas)

PÁGINA	ROTA	DESCRIÇÃO
AdminClientes	`/admin/clientes`	CRUD de prefeituras
AdminUsuarios	`/admin/usuarios`	Gestão de usuários e papéis

AdminPlanejamentos	`/admin/planejamentos`	Planejamentos de campo e drone
AdminDrones	`/admin/drones`	Cadastro e gestão de drones
AdminVoos	`/admin/voos`	Histórico de voos + badge config local
AdminRegioes	`/admin/regioes`	Regiões e bairros
AdminRiskPolicy	`/admin/risk-policy`	Política de risco pluviométrico
AdminPluvioRisco	`/admin/pluvio-risco`	Dashboard de risco de chuva
AdminPluvioOperacional	`/admin/pluvio-operacional`	Análise operacional pluvial
AdminSla	`/admin/sla`	Gestão de SLA e auditoria
AdminOperacoes	`/admin/operacoes`	Operações de campo em andamento
AdminHistoricoAtendimento	`/admin/historico-atendimento`	Histórico + pontos recorrentes
AdminQuotas	`/admin/quotas`	Quotas de uso por cliente
AdminMapaComparativo	`/admin/mapa-comparativo`	Heatmap A/B de dois levantamentos
AdminHeatmapTemporal	`/admin/heatmap-temporal`	Heatmap animado com slider temporal
AdminPainelMunicipios	`/admin/painel-municipios`	Comparativo entre municípios
AdminCasosNotificados	`/admin/casos`	Centro de notificações de casos
AdminCanalCidadao	`/admin/canal-cidadao`	QR Code + lista de denúncias

Operador (7 páginas)

PÁGINA	ROTA	DESCRIÇÃO
OperadorInicioTurno	`/operador/inicio`	Dashboard do agente: stats ciclo + tipo de atividade
OperadorListaImoveis	`/operador/imoveis`	Lista de imóveis com status + cadastro rápido
OperadorFormularioVistoria	`/operador/vistoria/:imovelId`	Stepper 5 etapas de vistoria
OperadorMapa	`/operador/mapa`	Mapa com rota otimizada (TSP)

OperadorLevantamentos	`/operador/levantamentos`	Lista de levantamentos atribuídos
OperadorNovoItemManual	`/operador/levantamentos/novo-item`	Cadastro manual de foco
OperadorUsuarios	`/operador/usuarios`	Gestão de usuários (supervisor)

Públicas / Especiais

PÁGINA	ROTA	DESCRIÇÃO
DenunciaCidadao	`/denuncia/:slug/:bairroId`	Denúncia pública sem autenticação
NotificadorRegistroCaso	`/notificador/registrar`	Registro de caso (UBS/hospital)
LandingPage	`/`	Página inicial pública

5.3 Rotas

```
/                          → LandingPage ou redirect
/login                     → Login
/reset-password            → ResetPassword
/trocar-senha              → TrocarSenha
/install                   → Install (PWA)
/denuncia/:slug/:bairroId  → DenunciaCidadao (público)

[Autenticadas]
/dashboard                 → Dashboard / redirect inteligente
/levantamentos             → Levantamentos
/mapa                      → MapaInspecao

[OperadorGuard]
/operador                  → OperadorPage
/operador/inicio           → OperadorInicioTurno
/operador/imoveis          → OperadorListaImoveis
/operador/vistoria/:id     → OperadorFormularioVistoria
/operador/mapa             → OperadorMapa
/operador/levantamentos    → OperadorLevantamentos
/operador/levantamentos/novo-item → OperadorNovoItemManual

[AdminOrSupervisorGuard]
/operador/usuarios         → OperadorUsuarios
```

```
[AdminGuard]
/admin/*                    → Páginas admin (18 rotas)
/notificador/registrar      → NotificadorRegistroCaso
```

5.4 Componentes de Domínio

DIRETÓRIO	COMPONENTES-CHAVE
`components/levantamentos/`	`ItemDetailPanel.tsx` — painel completo do foco (score YOLO, casos próximos, voz, falso positivo, checkin, ação)
`components/vistoria/`	`VistoriaEtapa1-5.tsx` — etapas do formulário de vistoria
`components/map-v3/`	HeatmapLayer, ClusterLayer, DetectionMetadataCards — mapa principal
`components/dashboard/`	CasosNotificadosWidget, PluvioRiskWidget, SlaWidget, QuotaBanner
`components/sla/`	SlaWidget, SlaEvolutionChart, SlaAlertBell, ConcluirSlaDialog
`components/operador/`	Componentes específicos do fluxo de campo

---

6. Módulos Funcionais

6.1 SLA Operacional

O SLA é gerado automaticamente por trigger quando um `levantamento_item` é criado. O prazo considera:

- Prioridade do item
- Configuração do cliente (`sla_config`)
- Override por região (`sla_config_regiao`)
- Feriados nacionais e municipais (`sla_feriados`)
- Horário comercial (segunda–sexta, 08h–18h)

- Fatores de risco climático

Ciclo de vida:

```
em_prazo → [prazo vence] → vencido → [escalonamento] → escalado
        → [resolução] → resolvido
```

Edge Functions relacionadas:

- `sla-marcar-vencidos` — executa a cada 15min via cron
- `sla-push-critico` — dispara Web Push quando SLA < 1h

6.2 Risco Pluviométrico

Score calculado diariamente por região com base em:

VARIÁVEL	IMPACTO
`chuva_mm`	Volume 24h
`dias_sem_chuva`	Janela seca (larvas em dev.)
`temperatura`	Ótimo 25–30°C
`vento`	Redutor acima 13km/h
`persistencia_7d`	Dias consecutivos com chuva
`tendencia`	Crescente (+5pp) / decrescente (−5pp)

> ***Janela crítica**: 3–6 dias após chuva intensa (desenvolvimento ativo de larvas).*

6.3 Vistoria de Campo (e-VISITA PNCD)

Fluxo de 5 etapas para registro completo de uma vistoria domiciliar:

```
Etapa 1 — Responsável
  ├─ GPS checkin automático (navigator.geolocation)
  ├─ Contador de moradores
  └─ Grupos vulneráveis (grávidas, idosos, crianças < 7 anos)
```

```
Etapa 2 — Sintomas
  ├─ 4 toggles de sintomas
  ├─ Contador de moradores afetados
  └─ Banner alerta: caso suspeito será gerado automaticamente

Etapa 3 — Inspeção de Depósitos
  └─ Para cada tipo A1–E: qtd inspecionados + com focos

Etapa 4 — Tratamento
  └─ Para depósitos com foco: eliminados + larvicida (g)

Etapa 5 — Riscos + Finalizar
  ├─ Riscos Social / Sanitário / Vetorial
  └─ FINALIZAR → status = 'visitado' → tela de confirmação
```

Ciclos: 6 ciclos anuais (bimestrais). Calculado como `Math.ceil((month + 1) / 2)`.

6.4 Centro de Notificações de Casos

Fluxo de Registro

1. Notificador (UBS/hospital) registra caso em `/notificador/registrar`
2. Geocodificação do endereço do paciente
3. INSERT em `casos_notificados`
4. **Trigger automático** cruza com `levantamento_itens` em raio 300m (PostGIS)
5. Itens próximos têm prioridade elevada para **Crítico**
6. Operador recebe alerta no `ItemDetailPanel`

Raios de Atuação

CONTEXTO	RAIO	AÇÃO
Cruzamento automático	300m	Eleva prioridade para Crítico
Alerta no ItemDetailPanel	300m	Banner vermelho para operador
Sugestão de planejamento	500m	3+ casos → botão criar levantamento

6.5 Análise IA Pós-Voo

Edge Function `triagem-ia-pos-voo` executa após um voo:

1. Agrupa focos por grade de 0,001° (~100m)
2. Filtra falsos positivos por score YOLO < 0,45
3. Envia clusters para Claude Haiku (sumário executivo)
4. Persiste resultado em `levantamento_analise_ia`

Feedback: Operador pode marcar item como "Não confirmado em campo" → grava em `yolo_feedback` para re-treino periódico.

6.6 Canal Cidadão

- Página pública `/denuncia/:slug/:bairroId` sem autenticação
- Formulário simples de denúncia
- Chama RPC `denuncia_cidadao()` com `SECURITY DEFINER`
- Grava como `levantamento_item` com `payload.fonte = 'cidadao'`
- Admin visualiza e gerencia em `/admin/canal-cidadao`

6.7 PWA e Suporte Offline

Service Worker (Workbox — `vite.config.ts`)

RECURSO	ESTRATÉGIA	CACHE
API Supabase REST	NetworkFirst, 5s timeout	24h, 200 entradas
Imagens Cloudinary	CacheFirst	7d, 300 entradas
Open-Meteo	NetworkFirst, 5s timeout	1h

Fila Offline (`src/lib/offlineQueue.ts`)

Operações IndexedDB que são drenadas automaticamente ao reconectar:

TIPO	AÇÃO
`checkin`	`api.itens.registrarCheckin`
`update_atendimento`	`api.itens.updateAtendimento`
`save_vistoria`	Sequência completa: create → depositos → sintomas → riscos

Banner `OfflineBanner` exibe contagem de operações pendentes no `AppLayout`.

---

7. Segurança

7.1 Row Level Security (RLS)

Todas as 37 tabelas têm RLS habilitado. Política padrão:

```
CREATE POLICY "isolamento_por_cliente" ON tabela
  USING (
    cliente_id IN (
      SELECT cliente_id FROM usuarios
      WHERE auth_id = auth.uid()
    )
  );
```

Auditoria: A view `public.view_rls_rules` (lê `pg_policies`) permite verificar todas as políticas ativas sem navegar no painel Supabase.

7.2 RBAC — Papéis

PAPEL	ROTAS ACESSÍVEIS
`admin`	Todas

`supervisor`	`/admin/*` (limitado), `/operador/*`
`usuario`	`/levantamentos`, `/mapa`, `/operador/*`
`operador`	`/operador/*`
`notificador`	`/notificador/registrar`

Implementado via RPC `get_meu_papel(auth_id)` + guards no React Router:

- `ProtectedRoute` — requer autenticação
- `OperadorGuard` — requer papel operador/usuario/supervisor/admin
- `AdminOrSupervisorGuard` — requer papel admin ou supervisor
- `AdminGuard` — requer papel admin

7.3 LGPD

MÓDULO	MEDIDA
Vistoria de campo	Apenas contagem de moradores — sem nome, CPF, data de nascimento
Casos notificados	Apenas endereço e bairro — sem identificação do paciente
Denúncia cidadão	Sem coleta de dados do denunciante
Evidências fotográficas	Armazenadas no Cloudinary sem associação a pessoas identificáveis

8. Histórico de Migrations

ARQUIVO	DATA	CONTEÚDO
`20250301000000`	01/03	Permite null em auth_id (usuarios)
`20250302000000`	02/03	RLS regioes + função helper
`20250302100000`	02/03	RLS master para todas as tabelas
`20250303000000`	03/03	SLA operacional + RLS

`20250306000000`	06/03	RLS operador e gestão de usuários
`20250306100000`	06/03	Evidências de atendimento + tipo_entrada
`20250306110000`	06/03	Trigger concluido_em em operacoes
`20250306120000`	06/03	RLS operacoes + evidencias
`20250306130000`	06/03	VIEW v_historico_atendimento_local
`20250306140000`	06/03	RPC get_meu_papel()
`20250306150000`	06/03	Permissões papeis_usuarios
`20250306160000`	06/03	Seed dados operador demo
`20250306170000`	06/03	RPC criar_levantamento_item_manual
`20250307100000`	07/03	Coluna tipo_levantamento em planejamento
`20250307120000`	07/03	Tabela levantamento_item_evidencias + RLS
`20250307140000`	07/03	Coluna observacao_atendimento
`20250307160000`	07/03	Enum status_atendimento
`20250308120000`	08/03	RPC criar_levantamento_item_manual v2
`20250308150000`	08/03	sla_calcular_prazo_final + trigger auto-SLA
`20250309100000`	09/03	Trigger SLA de pluvio_operacional_item
`20250309110000`	09/03	RPCs marcar_slas_vencidos + escalar_sla
`20250309120000`	09/03	sla_config_audit + trigger de auditoria
`20250309130000`	09/03	sla_operacional com levantamento_item_id
`20250311100000`	11/03	levantamento_item_status_historico + trigger + view
`20250311110000`	11/03	plano_acao_catalogo + seed
`20250311120000`	11/03	levantamento_item_recorrencia + trigger + view
`20250311130000`	11/03	sla_feriados + horário comercial no cálculo
`20250311140000`	11/03	v_slas_iminentes + RPC escalar_slas_iminentes
`20250311150000`	11/03	Checkin GPS em levantamento_itens
`20250311160000`	11/03	piloto_id em voos, config_fonte em levantamentos
`20250311170000`	11/03	RPC avaliar_condicoes_voo
`20250311180000`	11/03	sla_config_regiao + sla_resolve_config
`20250311190000`	11/03	cliente_quotas + v_cliente_uso_mensal

`20250311200000`	11/03	Coluna regiao_id em planejamento
`20250311210000`	11/03	Constraint fixes
`20250313000000`	13/03	tipo_item em plano_acao_catalogo
`20250317000000`	17/03	push_subscriptions (Web Push)
`20250317001000`	17/03	Canal cidadão (RPC SECURITY DEFINER)
`20250317002000`	17/03	levantamento_analise_ia + yolo_feedback (IA Sprint 4)
`20250318000000`	18/03	Centro de Notificações (unidades_saude, casos_notificados, cruzamento, PostGIS trigger)
`20250318001000`	18/03	Vistoria de Campo (imoveis, vistorias, depositos, sintomas, riscos, RPC resumo_agente)

Documentação gerada em Março 2026 — SentinelaWeb v1.0